LLM & RAG

Complete Theory Notes

Large Language Models • Retrieval-Augmented Generation

Architecture • Components • Techniques • Code • Best Practices

LLMs | Transformers | RAG | Vector DBs | Embeddings

Beginner to Advanced • 30-Page Comprehensive Guide • 2024 Edition

Table of Contents

PART 1 — Large Language Models (LLMs)

Theory, Architecture, Training, and Practical Usage

1.1 What is a Large Language Model?

A **Large Language Model (LLM)** is an AI model trained on massive amounts of text to understand and generate human language. The word *large* refers to billions of learnable parameters and training on hundreds of billions of tokens. LLMs are **foundation models** — general-purpose models adaptable to many tasks without task-specific retraining.

LLMs power: question answering, code generation, summarization, translation, reasoning, creative writing, and more — all from a single model trained on diverse text data.

Key Characteristics of LLMs

- Scale: billions of parameters (GPT-4 ~1.7T, Llama 3 70B, Gemini Ultra ~1.5T estimated)
- Training data: web text, books, code, papers — trillions of tokens of diverse text
- Self-supervised: trained to predict next token — no human labels needed for pre-training
- Emergent capabilities: abilities that appear only at large scale (reasoning, coding, math)
- In-context learning: learn new tasks just from examples given in the prompt — no retrainin
- Generalization: one model handles thousands of different task types seamlessly

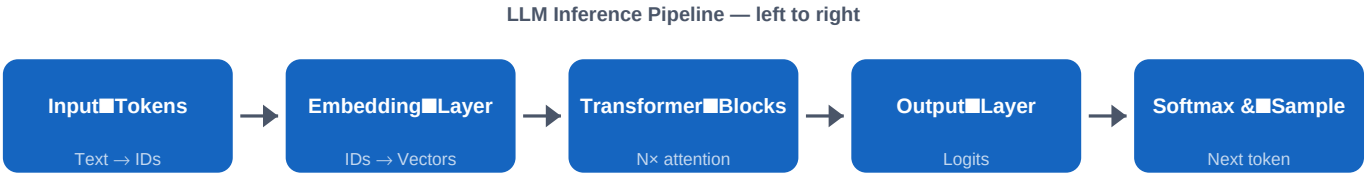
1.2 History & Evolution of LLMs

Year	Model / Event	Significance
2013	Word2Vec (Google)	First practical word embeddings — semantic vector representations
2017	Transformer (Google)	Attention is All You Need — the foundational architecture
2018	BERT (Google)	Bidirectional pre-training, the fine-tuning paradigm
2018	GPT-1 (OpenAI)	Generative pre-training on language modeling task
2019	GPT-2 (OpenAI)	1.5B params — initially held back as 'too dangerous'
2020	GPT-3 (OpenAI)	175B params — few-shot learning breakthrough moment
2022	ChatGPT (OpenAI)	RLHF alignment — conversational AI goes mainstream globally
2023	GPT-4, Llama 2, Claude 2	Multi-modal models, open-source era begins
2024	Llama 3, Gemini 1.5, Claude 3	Long context windows, open weights, multimodal standard

1.3 How LLMs Work — The Big Picture

LLMs learn statistical patterns in language through **next-token prediction**. Given a sequence of tokens, the model predicts what comes next. Doing this billions of times across massive datasets, the model internalizes grammar, facts, and reasoning patterns.

At inference time the model takes your **prompt**, processes it through many neural network layers, and outputs a probability distribution over all vocabulary tokens. The top token is sampled and appended — then the process repeats. This is called **autoregressive generation**.



1.4 Transformer Architecture

The **Transformer** (2017, Google) is the neural network underlying virtually all modern LLMs. It replaced RNNs with a purely attention-based mechanism that processes all tokens in **parallel** — enabling massive scaling.

Encoder (BERT-style)

- Processes input bidirectionally
- Sees context from left AND right
- Good for: classification, NER, QA
- Examples: BERT, RoBERTa, DeBERTa
- Output: contextualized embeddings

Decoder (GPT-style)

- Generates tokens one at a time
- Only sees tokens to the LEFT (causal)
- Good for: text generation, chat
- Examples: GPT-4, Llama, Claude, Gemini
- Output: probability over vocabulary

Key Transformer Components:

Token Embeddings — Each token ID maps to a dense vector (e.g. 4096 dimensions). Learned during training, these capture semantic relationships between tokens.

Positional Encoding — Adds position information to token vectors. Modern models use RoPE (Rotary Position Embedding) or ALiBi for better long-context handling.

Multi-Head Self-Attention — Core mechanism. Each token attends to all others, computing a weighted sum of values. Multiple heads capture different relationship types simultaneously.

Feed-Forward Network (FFN) — 2-layer MLP applied after attention. Most factual knowledge is believed to be stored in FFN weights.

Layer Norm + Residual Connections — Normalize activations and add skip connections. Critical for training stability in deep networks (32–96+ layers).

1.5 Attention Mechanism

Attention lets the model focus on relevant parts of the input when processing each token. It answers: *for this token, which other tokens matter most?* Each token is projected into three vectors — **Query (Q)**, **Key (K)**, **Value (V)** — and attention scores are dot products of Q and K, scaled and softmaxed into weights for V.

Types of Attention in Modern LLMs

- Full Self-Attention — every token attends to every other, O(n²) — expensive for long conte
- Causal (Masked) Attention — each token only attends LEFT — used in all GPT-style decoders
- Multi-Head Attention (MHA) — H parallel heads, each learns different relationship patterns
- Grouped Query Attention (GQA) — multiple Q heads share K/V pairs — reduces memory (Llama 3
- Flash Attention — hardware-optimized implementation, same result, much faster & less VRAM
- Sliding Window Attention — each token attends to a local window only (Mistral, Gemma model

Context Window — the maximum number of tokens a model can attend to at once. GPT-3: 4K. GPT-4 Turbo: 128K. Claude 3: 200K. Gemini 1.5 Pro: 1M tokens.

1.6 LLM Training Process

Training an LLM happens in multiple phases:

Phase 1 — Pre-training (Self-Supervised)

- Data: massive crawl of internet text, books, code — trillions of tokens
- Objective: next-token prediction (cross-entropy loss) — no human labels needed
- Scale: thousands of GPUs (H100/A100) running for weeks or months
- Cost: $1M–$100M+ for frontier models
- Output: a base model that completes text but doesn't follow instructions well

Phase 2 — Supervised Fine-Tuning (SFT)

- Data: human-written instruction-response pairs (10K–1M high-quality examples)
- Objective: teach the model to follow instructions and be a helpful assistant
- Much cheaper than pre-training — days on far fewer GPUs
- Output: an instruction-following model (base ChatGPT, Claude, Gemini)

Phase 3 — RLHF / DPO (Alignment)

- RLHF: train a Reward Model on human preference pairs, then PPO to maximize reward
- DPO: Direct Preference Optimization — simpler alternative, no RL loop needed
- Goal: make model more Helpful, Harmless, and Honest (HHH alignment)
- Output: the aligned chat model — safe, helpful, follows conversational norms

1.7 Tokenization

Tokenization converts raw text into integer IDs the model can process. LLMs work with **tokens** — sub-word units. Vocabulary size: 32K–128K tokens. 1 token ≈ 0.75 English words.

Common Tokenizers

- BPE — Byte Pair Encoding (GPT-4, GPT-2)
- WordPiece — BERT, DistilBERT
- SentencePiece — Llama, T5, Gemini
- tiktoken — OpenAI's fast BPE library
- 1 token ≈ 0.75 English words
- ~100 tokens ≈ 75 words ≈ 1 paragraph

Tokenization Facts

- 'Hello World' = 2 tokens (GPT-4)
- 'ChatGPT' = 3 tokens: Chat|G|PT
- Code is more token-dense than prose
- Non-English text = more tokens/word
- Max tokens = your context window limit
- Count tokens BEFORE sending to API

Python

```
# Token counting with tiktoken
pip install tiktoken

import tiktoken
enc = tiktoken.encoding_for_model('gpt-4o')

text = 'Hello, how are you today?'
tokens = enc.encode(text)
print(tokens) # [9906, 11, 1268, 527, 499, 3432, 30]
print(len(tokens)) # 7 tokens

# Useful helper — always count before expensive API calls
def count_tokens(text: str, model='gpt-4o') -> int:
    enc = tiktoken.encoding_for_model(model)
    return len(enc.encode(text))

print(count_tokens('Explain quantum computing in detail.'))
```

1.8 Embeddings — Semantic Vector Representations

An **embedding** is a dense float vector representing the semantic meaning of text. Similar concepts have similar vectors. Classic example: *king* – *man* + *woman* ≈ *queen*. Used for: semantic search, RAG retrieval, clustering, classification, and recommendation.

Popular Embedding Models

- text-embedding-3-small (OpenAI) — 1536 dims, $0.02/1M tokens, excellent quality
- text-embedding-3-large (OpenAI) — 3072 dims, best OpenAI quality for complex tasks
- all-MiniLM-L6-v2 (HuggingFace) — 384 dims, FREE, fast, great for local use
- all-mpnet-base-v2 (HuggingFace) — 768 dims, FREE, best open-source quality
- embed-english-v3.0 (Cohere) — 1024 dims, strong retrieval performance
- nomic-embed-text (Ollama) — 768 dims, FREE, runs locally, no API key required

Cosine Similarity — angle between vectors (0=unrelated, 1=identical). Most common metric for semantic search and RAG retrieval.

Dimensions — higher = richer representation but more storage/compute. 384–1536 dims is the practical range. Match model dimension to your vector DB index.

Critical Rule — always use the SAME embedding model for both indexing documents and embedding queries. Mixing models breaks similarity scores.

1.9 Inference & Decoding Strategies

The model outputs a probability distribution over vocabulary at each step. The **decoding strategy** determines how to sample from this distribution.

Strategy	How It Works	Use Case
Greedy	Always pick highest probability token	Fast, deterministic, often repetitive
Temperature	Scale logits (T<1=focused, T>1=random)	T=0 for facts, T=0.7-1.0 for creative
Top-K	Sample from top K tokens only	Quality + diversity balance (K=40-50)
Top-P (Nucleus)	Sample from smallest set summing to P	Best for most tasks (P=0.9 typical)
Beam Search	Explore multiple sequences simultaneously	Translation, structured generation
Repetition Penalty	Reduce prob of already-generated tokens	Prevents repetitive loops in output

Key params: temperature=0 (deterministic/factual), 0.7 (balanced), 1.0+ (creative). max_tokens limits output length. stop_sequences halt generation early.

1.10 Prompt Engineering

Prompt engineering crafts inputs to elicit the best LLM outputs. Since LLMs are sensitive to phrasing and structure, well-designed prompts dramatically improve quality.

Core Prompting Techniques

- Zero-shot: direct question with clear role and format instructions
- Few-shot: provide 2-5 input→output examples before your actual question
- Chain-of-Thought (CoT): ask to 'think step by step' before giving the final answer
- System prompt: set role, tone, constraints, and behavior at conversation start
- ReAct: Reasoning + Acting — interleave thought/action/observation steps for agents
- Self-consistency: generate N answers, pick the most common (majority vote improves accurac
- Tree-of-Thoughts: explore multiple reasoning paths, backtrack when needed
- Role prompting: 'You are an expert Python developer with 10 years experience...'

Python

```python
# Few-shot prompting example
few_shot = '''Classify sentiment of reviews:

Review: 'This product is amazing!' -> Positive
Review: 'Broke after one day, terrible.' -> Negative
Review: 'Works fine, nothing special.' -> Neutral

Review: 'Best purchase I have ever made!' ->
'''

# Chain-of-Thought prompting
cot_prompt = '''Solve this problem step by step.
Problem: A train travels 120km in 2 hours. What is its speed?
Step 1: Identify known values
Step 2: Apply formula speed = distance / time
Step 3: Calculate: 120 / 2 = ?
Answer:'''
```

1.11 Fine-Tuning LLMs

Fine-tuning adapts a pre-trained LLM to a specific domain by continuing training on a smaller curated dataset, updating weights for your task.

Full Fine-Tuning

- Updates ALL model parameters
- Requires large GPU memory (80GB+)
- Most powerful — best task results
- Cost: $1K–$100K+ for large models
- Risk of catastrophic forgetting
- Use for: domain-critical applications

PEFT / LoRA / QLoRA

- Trains only a small subset of params
- LoRA: adds low-rank adapter matrices
- QLoRA: LoRA + 4-bit quantization
- Runs on consumer GPUs (24GB VRAM)
- 90-99% fewer trainable parameters
- Use for: most practical fine-tuning

Fine-Tune When: you need specific style/tone/format, the task requires fixed domain knowledge, or latency is critical and you can't afford retrieval.

Use RAG When: knowledge changes frequently, you need citations/sources, or you have a large and growing knowledge base.

1.12 LLM Providers & Models Reference

Provider	Model	Params	Context	Strengths
OpenAI	GPT-4o	~200B	128K	Best overall, multimodal, fast
OpenAI	GPT-4o-mini	~8B	128K	Fast, cheap, excellent quality/cost
Anthropic	Claude 3.5 Sonnet	~70B?	200K	Coding, long context, safety
Anthropic	Claude 3 Haiku	~7B?	200K	Fastest Claude, very low cost
Google	Gemini 1.5 Pro	~540B?	1M	Longest context, multimodal
Google	Gemini 1.5 Flash	~8B?	1M	Fast and affordable Gemini
Meta	Llama 3.1 70B	70B	128K	Best open-source, completely free
Meta	Llama 3.2 3B	3B	128K	Runs on CPU/mobile devices
Mistral	Mixtral 8x7B	56B MoE	32K	MoE architecture, fast, open-source
Microsoft	Phi-3 Mini	3.8B	128K	Tiny but powerful, runs locally

1.13 LLM Evaluation

Evaluation Dimensions & Metrics

- Perplexity — how well model predicts held-out test text (lower = better language model)
- BLEU / ROUGE — n-gram overlap with reference answers (translation, summarization tasks)
- BERTScore — semantic similarity via embeddings (better than exact-match metrics)
- Accuracy — on classification/QA benchmarks: MMLU, HellaSwag, GSM8K, HumanEval
- Human Eval — human judges rate helpfulness, accuracy, safety on 1-5 scale
- LLM-as-Judge — use GPT-4 to evaluate outputs of other models (scalable, correlates well)
- MT-Bench — multi-turn conversation benchmark, GPT-4 judges across 8 categories
- Arena ELO — crowdsourced A/B battles at LMSys Chatbot Arena leaderboard

1.14 Limitations & Challenges

Technical Limitations

- Hallucination — confidently wrong facts
- Knowledge cutoff — no real-time info
- Context window — limited input size
- High latency for large frontier models
- Expensive API costs at production scale
- Inconsistent math/reasoning performance
- Prompt sensitivity — rephrasing changes output

Safety & Alignment Issues

- Jailbreaking — bypassing safety guardrails
- Bias — reflects training data biases
- Sycophancy — agreeing vs being honest
- Prompt injection — malicious inputs
- Privacy — may memorize training data
- Copyright — training data concerns
- Misuse — deepfakes, disinformation

PART 2 — Retrieval-Augmented Generation (RAG)

Architecture, Components, Techniques, Evaluation, and Production

2.1 What is RAG & Why Does It Matter?

Retrieval-Augmented Generation (RAG) enhances LLMs by connecting them to external knowledge at inference time. Instead of relying on knowledge baked into weights during training, RAG *retrieves* relevant documents and *injects* them as context before generation.

RAG was introduced by Facebook AI (2020) to overcome the fundamental limitation of LLMs: their knowledge is frozen at training time. Today it is the **dominant architecture** for enterprise AI applications requiring accurate, up-to-date, verifiable answers.

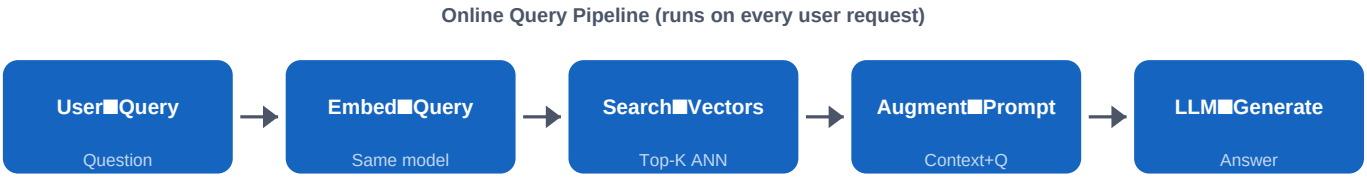
Why RAG? — Problems It Solves

- Knowledge cutoff — LLM training data ends at date X; RAG provides fresh documents always
- Hallucination — grounding in retrieved facts dramatically reduces confabulated answers
- Verifiability — show users exactly which source documents the answer came from
- Cost efficiency — update knowledge base without expensive model retraining at all
- Privacy — sensitive data stays in your own vector store, never in model weights
- Domain specificity — inject proprietary docs (manuals, contracts, policies) at query time

2.2 RAG Architecture — Full System Overview

OFFLINE PHASE — Building the Index:

ONLINE PHASE — Answering Queries:



The 8-Step RAG Pipeline

- 1. LOAD — Read documents from PDFs, URLs, databases, APIs, S3, Notion, etc.
- 2. SPLIT — Chunk documents into overlapping segments (e.g. 1000 chars, 200 overlap)
- 3. EMBED — Convert each chunk to a dense vector using an embedding model
- 4. STORE — Persist vectors + chunk text in a vector database (Chroma, FAISS, Pinecone)
- 5. QUERY EMBED — Convert user's question to vector using the SAME embedding model
- 6. RETRIEVE — Find top-K most similar chunks via approximate nearest neighbor search
- 7. AUGMENT — Build prompt: [system instructions] + [retrieved context] + [user question]
- 8. GENERATE — Send augmented prompt to LLM, stream the grounded answer back to user

2.3 Document Processing — Loading & Splitting

Document Loaders read data from various sources producing Document objects (text + metadata). **Text Splitters** divide documents into chunks small enough to embed and retrieve efficiently.

Document Loaders

- PyPDFLoader — PDF files
- WebBaseLoader — HTML web pages
- TextLoader — plain .txt files
- CSVLoader — CSV, each row = doc
- JSONLoader — JSON with jq paths
- DirectoryLoader — entire folders
- UnstructuredLoader — DOCX, PPTX
- YoutubeLoader — video transcripts

Splitting Strategies

- RecursiveCharacterSplitter (best)
- Tries: \n\n → \n → space → char
- chunk_size=1000 (chars or tokens)
- chunk_overlap=200 (context bridge)
- MarkdownHeaderSplitter — by headers
- TokenTextSplitter — token-based
- SemanticSplitter — by meaning shift
- HTMLSectionSplitter — by HTML tags

chunk_size: 500–2000 chars. Smaller = more precise retrieval. Larger = more context per chunk. Match to expected answer length.

chunk_overlap: 10–20% of chunk_size. Prevents losing information at chunk boundaries — critical for long sentences spanning chunks.

Metadata enrichment: add source, page number, section, date to each chunk — enables metadata-filtered retrieval for precise results.

2.4 Embeddings in RAG

Embeddings convert document chunks and queries into vectors in the same semantic space, enabling similarity-based retrieval. **Use the SAME embedding model for indexing and querying.**

Python

```python
from langchain_openai import OpenAIEmbeddings
from langchain_community.embeddings import HuggingFaceEmbeddings

# Option A: OpenAI (paid, high quality)
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
# 1536 dimensions — best quality for production

# Option B: HuggingFace (FREE — no API key!)
hf_emb = HuggingFaceEmbeddings(
    model_name='sentence-transformers/all-MiniLM-L6-v2',
    model_kwargs={'device': 'cpu'},
    encode_kwargs={'normalize_embeddings': True}
)

# Embed a query
vector = embeddings.embed_query('What is machine learning?')
print(f'Dimensions: {len(vector)}') # 1536

# Embed documents (batched)
texts = ['Doc one content.', 'Doc two content.']
vecs = embeddings.embed_documents(texts)

# Manual cosine similarity
import numpy as np
def cosine(a, b):
    return np.dot(a,b) / (np.linalg.norm(a)*np.linalg.norm(b))
score = cosine(vector, vecs[0])
print(f'Similarity: {score:.4f}') # 0.0 to 1.0
```

2.5 Vector Databases

A vector database stores embeddings alongside source text and metadata, providing fast **Approximate Nearest Neighbor (ANN)** search to find the most similar vectors at scale.

How Vector Search Works

- Indexing: build an ANN index (HNSW, IVF, LSH) over all stored vectors
- HNSW — graph-based index, very fast query speed, high memory usage
- IVF — clustering-based, scalable to billions, slight accuracy trade-off
- Query: embed the query → search ANN index → return top-K closest vectors
- Distance metrics: Cosine (angle), Dot Product, Euclidean L2 distance
- Metadata filtering: filter by source/date/category BEFORE or AFTER vector search

Vector DB	Type	Best For	Key Fact
FAISS	In-memory/file	Local dev, fast prototyping	Meta OSS, pip install faiss-cpu
Chroma	Local/server	Dev to small production	pip install chromadb, simple API
Pinecone	Cloud managed	Production at scale	Serverless, fast, paid service
Weaviate	Cloud/self-host	Hybrid BM25 + vector	GraphQL API, open source
Qdrant	Cloud/self-host	High performance apps	Rust-based, fast filtering
Milvus	Cloud/self-host	Enterprise 1B+ vectors	Most scalable, complex setup
pgvector	PostgreSQL ext.	Existing PG infrastructure	SQL + vectors, no new service

2.6 Retrieval Strategies

Retrieval Strategies — From Basic to Advanced

- Similarity Search — cosine/dot product against all vectors (default, works well)
- MMR (Maximal Marginal Relevance) — balance relevance AND diversity, avoid redundant chunks
- Score Threshold — only return chunks above a minimum similarity score cutoff
- Metadata Filtering — filter by source/date/category before vector search (fast & targeted)
- Hybrid Search — combine dense (vector) + sparse (BM25/keyword) search + rerank results
- Multi-Query — LLM generates N phrasings of the query, retrieves for each, deduplicates
- HyDE — Hypothetical Document Embeddings: LLM generates hypothetical answer, embed & retrie
- Parent-Document — retrieve small child chunks, return their larger parent for richer conte
- Contextual Compression — LLM extracts only relevant sentences from retrieved chunks
- Re-ranking — cross-encoder model re-ranks retrieved chunks by relevance to query

Python

```python
# Similarity — basic
r1 = vectorstore.as_retriever(search_kwargs={'k': 4})

# MMR — diverse results
r2 = vectorstore.as_retriever(search_type='mmr',
    search_kwargs={'k':4,'fetch_k':20,'lambda_mult':0.5})

# Hybrid: BM25 keyword + vector
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever
bm25 = BM25Retriever.from_documents(chunks); bm25.k=4
hybrid = EnsembleRetriever(retrievers=[bm25, r1], weights=[0.4, 0.6])
```

2.7 Generation Phase — Augmented Prompting

Retrieved documents are injected into the prompt as context. The quality of **context formatting** and **prompt design** are critical for answer quality and grounding.

Python

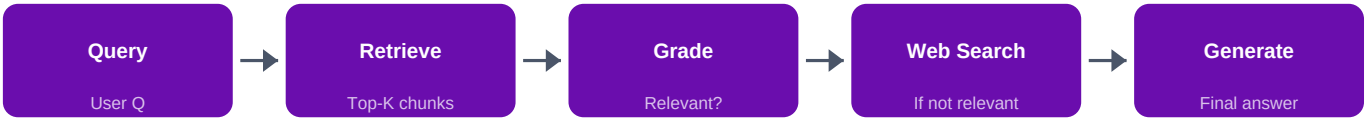
```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough, RunnableParallel
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

RAG_TEMPLATE = '''You are a helpful AI assistant.
Answer the question using ONLY the provided context.
If the answer is not in the context, say: I don't have enough
information in the provided documents to answer this.
Always cite which document you used.

Context:
{context}

Question: {question}
Answer:'''

prompt = ChatPromptTemplate.from_template(RAG_TEMPLATE)

def format_docs(docs):
    parts = []
    for i, doc in enumerate(docs):
        src = doc.metadata.get('source', 'Unknown')
        parts.append(f'[Doc {i+1}] Source: {src}')
        parts.append(doc.page_content)
    return '\n\n'.join(parts)

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

rag_chain = (
    RunnableParallel({'context': retriever | format_docs,
                      'question': RunnablePassthrough()})
    | prompt | llm | StrOutputParser()
)

for chunk in rag_chain.stream('What is the main topic?'):
    print(chunk, end='', flush=True)
```

2.8 Advanced RAG Patterns

Advanced RAG Techniques

- Query Rewriting — LLM rewrites user query for better retrieval accuracy before search
- Query Decomposition — break complex question into sub-questions, answer each separately
- Corrective RAG (CRAG) — grade retrieved docs; fall back to web search if irrelevant
- Self-RAG — LLM decides WHEN to retrieve, then critiques and refines its own output
- Adaptive RAG — classify query complexity, use simple/complex RAG path accordingly
- RAG Fusion — multi-query + Reciprocal Rank Fusion for superior result ranking
- GraphRAG (Microsoft) — extract knowledge graph from docs, traverse for complex queries
- RAPTOR — recursive summarization tree for hierarchical multi-level retrieval

Corrective RAG (CRAG) — adds relevance grading + web search fallback



2.9 RAG Evaluation — RAGAs Framework

Evaluating RAG requires measuring multiple dimensions: did we retrieve the right documents? Does the answer match the context? Is it factually correct?

RAGAs Core Metrics (pip install ragas)

- Faithfulness — does the answer contain ONLY info from retrieved context? (0-1 score)
- Answer Relevance — how relevant is the answer to the original question? (0-1 score)
- Context Recall — what % of ground-truth answer is covered by retrieved context? (0-1)
- Context Precision — what % of retrieved context is actually relevant to the question? (0-1
- Answer Correctness — is the answer factually correct vs ground truth? (requires GT labels)

Python

```
pip install ragas

from ragas import evaluate
from ragas.metrics import (
   faithfulness, answer_relevancy,
   context_recall, context_precision
)
from datasets import Dataset

eval_data = {
   'question': ['What is RAG?', 'How does FAISS work?'],
   'answer': ['RAG is...', 'FAISS is...'],
   'contexts': [['doc1','doc2'], ['doc1']],
   'ground_truth': ['RAG is a technique...', 'FAISS is a library...'],
}
result = evaluate(
   Dataset.from_dict(eval_data),
   metrics=[faithfulness, answer_relevancy,
          context_recall, context_precision]
)
print(result) # faithfulness:0.92 answer_relevancy:0.88
```

2.10 Production RAG Pipeline

Python

```python
# Production RAG — Part A: Load, Split, Embed, Store
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
import os
os.environ['OPENAI_API_KEY'] = 'sk-...'

# 1. Load
loader = PyPDFLoader('company_handbook.pdf')
docs = loader.load()
print(f'Loaded {len(docs)} pages')

# 2. Split
splitter = RecursiveCharacterTextSplitter(
   chunk_size=1000, chunk_overlap=200)
chunks = splitter.split_documents(docs)
print(f'Created {len(chunks)} chunks')

# 3. Embed + Store
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vectorstore = Chroma.from_documents(
   documents=chunks, embedding=embeddings,
   persist_directory='./prod_db'
)
```

Python

```python
# Production RAG — Part B: Retrieve and Generate
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough, RunnableParallel
from langchain_core.output_parsers import StrOutputParser

# 4. Create retriever (MMR for diverse results)
retriever = vectorstore.as_retriever(
    search_type='mmr',
    search_kwargs={'k': 5, 'fetch_k': 20}
)

# 5. RAG prompt
prompt = ChatPromptTemplate.from_template(
    'Answer using ONLY the context. Say I dont know if unsure.'
    '\nContext: {context}\nQuestion: {question}\nAnswer:'
)

# 6. Build LCEL chain
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
fmt = lambda docs: '\n---\n'.join(d.page_content for d in docs)

rag = (
    RunnableParallel({'context': retriever | fmt,
                      'question': RunnablePassthrough()})
    | prompt | llm | StrOutputParser()
)

# 7. Query
for token in rag.stream('What is the leave policy?'):
    print(token, end='', flush=True)
```

2.11 RAG vs Fine-Tuning

Criterion	RAG	Fine-Tuning
Knowledge updates	Easy — update vector store	Hard — retrain or re-fine-tune model
Knowledge type	Factual, document-grounded	Style, format, domain adaptation
Citations / sources	Yes — can show source documents	No — baked into model weights
Setup cost	Low — no GPU needed	High — GPU hours/dollars required
Inference cost	Slightly higher (retrieval step)	Same as base model
Hallucination risk	Low — grounded in context	Still possible — weights can confabulate
Privacy	Data stays in your vector DB	Data is used in the training process
Best use case	Q&A, document chat, search	Specific tone, task format, code style

Recommendation: Start with RAG. Faster to set up, easier to update, works for most enterprise use cases. Add fine-tuning only when RAG alone is insufficient.

2.12 RAG Best Practices

Document Processing

- Clean docs: remove headers, footers
- Use semantic chunking for better context
- Add rich metadata: source, date, section
- Test chunk sizes: 500, 1000, 2000 chars
- Use parent-doc retrieval for precision
- Re-index when docs change significantly

Retrieval Quality

- Use MMR to avoid redundant chunks
- Try hybrid search (BM25 + vector)
- Add a cross-encoder reranker
- Set similarity threshold, filter noise
- Retrieve more (k=10), rerank to 4
- Monitor retrieval with RAGAs metrics

Prompt Design

- Explicitly say: use ONLY context
- Instruct to say I don't know
- Ask for source citations always
- Include conversation history
- Specify output format explicitly
- Test with adversarial questions

Production Tips

- Cache embeddings to save costs
- Use async for parallel retrieval
- Log every query + retrieved docs
- Set alerts for low relevance scores
- A/B test retrieval strategies
- Version your vector store indexes

PART 3 — Code Reference & Cheat Sheet

Ready-to-use snippets, installation commands, and key concepts

3.1 LLM API — Multi-Provider Code

Python

```python
# ■■ OPENAI ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from openai import OpenAI
client = OpenAI(api_key='sk-...')
response = client.chat.completions.create(
   model='gpt-4o-mini',
   messages=[
      {'role':'system','content':'You are a helpful assistant.'},
      {'role':'user','content':'Explain transformers briefly.'}
   ],
   temperature=0.7, max_tokens=500
)
print(response.choices[0].message.content)
print(f'Tokens: {response.usage.total_tokens}')

# ■■ ANTHROPIC CLAUDE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import anthropic
client = anthropic.Anthropic(api_key='sk-ant-...')
msg = client.messages.create(
   model='claude-3-5-sonnet-20241022', max_tokens=1024,
   system='You are a helpful assistant.',
   messages=[{'role':'user','content':'Explain RAG simply'}]
)
print(msg.content[0].text)

# ■■ GOOGLE GEMINI ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import google.generativeai as genai
genai.configure(api_key='AIza...')
model = genai.GenerativeModel('gemini-1.5-flash')
print(model.generate_content('Explain attention mechanism').text)

# ■■ OLLAMA (local, completely free) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import ollama
response = ollama.chat(model='llama3.2',
   messages=[{'role':'user','content':'What is LangChain?'}])
print(response['message']['content'])
```

3.2 Complete RAG with FAISS + HuggingFace (100% Free!)

Python

```python
# pip install langchain langchain-community faiss-cpu
# pip install sentence-transformers ollama

from langchain_community.document_loaders import WebBaseLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

# Load a web page (no API key needed)
loader = WebBaseLoader('https://en.wikipedia.org/wiki/Transformer_(machine_learning)')
docs = loader.load()

# Split
chunks = RecursiveCharacterTextSplitter(
    chunk_size=800, chunk_overlap=150
).split_documents(docs)

# FREE embeddings — no API key needed
emb = HuggingFaceEmbeddings(
    model_name='sentence-transformers/all-MiniLM-L6-v2'
)

# Build FAISS vector store
vs = FAISS.from_documents(chunks, emb)
retriever = vs.as_retriever(search_kwargs={'k': 4})

# FREE local LLM (run: ollama pull llama3.2)
llm = ChatOllama(model='llama3.2')

fmt = lambda docs: '\n\n'.join(d.page_content for d in docs)
chain = (
    {'context': retriever | fmt, 'question': RunnablePassthrough()}
    | ChatPromptTemplate.from_template(
        'Context: {context}\nQuestion: {question}\nAnswer:')
    | llm | StrOutputParser()
)
print(chain.invoke('What is the attention mechanism?'))
```

3.3 Installation Reference

Shell

```
# ■■ CORE LLM SDKs ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pip install openai # OpenAI Python SDK
pip install anthropic # Anthropic Claude SDK
pip install google-generativeai # Google Gemini SDK
pip install ollama # Ollama local LLM client
pip install transformers torch # HuggingFace + PyTorch

# ■■ LANGCHAIN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pip install langchain langchain-core langchain-community
pip install langchain-openai langchain-anthropic langchain-ollama

# ■■ EMBEDDINGS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pip install sentence-transformers # Free HuggingFace embeddings
pip install tiktoken # Token counting (OpenAI)

# ■■ VECTOR STORES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pip install faiss-cpu # FAISS (in-memory, local)
pip install chromadb # Chroma (local persistent)
pip install pinecone-client # Pinecone (cloud managed)
pip install qdrant-client # Qdrant (cloud/self-host)

# ■■ DOCUMENT PROCESSING ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pip install pypdf # PDF loading
pip install unstructured # DOCX, HTML, PPTX loading
pip install beautifulsoup4 lxml # Web scraping support

# ■■ EVALUATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pip install ragas # RAGAs evaluation framework

# ■■ UTILITIES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pip install python-dotenv # .env file support
pip install fastapi uvicorn # Serve RAG as REST API
```

Key Concepts Glossary

Term	Definition
Token	Sub-word unit, ~0.75 words. LLMs process token sequences not raw text.
Parameter	Learnable weight in neural network. GPT-4 has ~1.7T parameters.
Context Window	Max tokens model can see at once. Ranges from 4K (GPT-3) to 1M (Gemini).
Temperature	Randomness of sampling. 0=deterministic, 0.7=balanced, 1.2=very creative.
Embedding	Dense float vector representing semantic meaning of text in N dimensions.
Cosine Similarity	Angle-based vector similarity. 1.0=identical meaning, 0.0=unrelated.
Chunking	Splitting documents into smaller overlapping pieces for indexing and retrieval.
ANN Search	Approximate Nearest Neighbor — fast vector search (HNSW, IVF algorithms).

Hallucination	LLM generating plausible-sounding but factually incorrect information.
RLHF	Reinforcement Learning from Human Feedback — key alignment training technique.
LoRA	Low-Rank Adaptation — efficient fine-tuning by adding small adapter matrices.
RAG	Retrieval-Augmented Generation — combine retrieval with LLM generation.
MMR	Maximal Marginal Relevance — retrieval balancing relevance and diversity.
Perplexity	Model uncertainty on text prediction. Lower = better language model.
RAGAs	Framework to evaluate RAG systems: Faithfulness, Relevance, Recall, Precision.

Official Documentation & Learning Resources:

OpenAI: platform.openai.com/docs • HuggingFace: huggingface.co/docs • LangChain: python.langchain.com

RAGAs: docs.ragas.io • Chroma: docs.trychroma.com • FAISS: faiss.wiki.kernel.org

Paper: Attention is All You Need — arxiv.org/abs/1706.03762 • RAG Paper — arxiv.org/abs/2005.11401

Courses: fast.ai • deeplearning.ai • Andrej Karpathy (YouTube) • HuggingFace NLP Course

BONUS — Fine-Tuning, LoRA & Production Deployment

Adapt models to your domain and ship to production

Fine-Tuning Techniques Comparison

Fine-tuning adapts a pre-trained LLM to a specific task by continuing training on a smaller curated dataset. Modern approaches use **parameter-efficient fine-tuning (PEFT)** which trains only a tiny fraction of weights — enabling fine-tuning on consumer GPUs.

	GPU Memory	When to Use
	Very High	Max accuracy, have GPU cluster
	Low	Most common — best accuracy/cost balance
	Very Low	Consumer GPU (RTX 3090/4090)
	Very Low	Task-specific prefix token tuning
	Minimal	Large models, lightweight adaptation
	Low	Modular, task-specific adapters

Python

```python
# pip install transformers datasets peft trl accelerate bitsandbytes

from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, TaskType
from trl import SFTTrainer, SFTConfig

# Load in 4-bit quantization (QLoRA — fits on 24GB GPU)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype='float16'
)
model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Meta-Llama-3-8B',
    quantization_config=bnb_config, device_map='auto'
)

# LoRA: inject small trainable matrices A*B into attention layers
lora_config = LoraConfig(
    r=16, # rank — higher=more capacity
    lora_alpha=32, # scaling factor (usually 2*r)
    target_modules=['q_proj', 'v_proj', 'k_proj', 'o_proj'],
    lora_dropout=0.05, task_type=TaskType.CAUSAL_LM
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters() # trainable: 0.42% of 8B params

# Fine-tune with SFT (Supervised Fine-Tuning)
trainer = SFTTrainer(
    model=model,
    args=SFTConfig(output_dir='./ft_output', num_train_epochs=3,
                per_device_train_batch_size=4, learning_rate=2e-4),
    train_dataset=dataset, # dataset with 'text' or 'messages' column
)
trainer.train()
model.save_pretrained('./my_fine_tuned_model')
```

RAG API Deployment with FastAPI

Serving Options

- OpenAI/Anthropic API — zero infra
- vLLM — high-throughput OSS inference
- Ollama — local, privacy-first serving
- HuggingFace TGI — production OSS
- Replicate — serverless model hosting
- AWS Bedrock — managed multi-provider

Production Checklist

- Rate limiting — throttle per API key
- Streaming SSE — responsive UX
- Semantic caching — skip repeat queries
- Retry with backoff — handle 429 errors
- Cost tracking — monitor token spend
- LangSmith tracing — debug every call

Python

```python
from fastapi import FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel

app = FastAPI(title='RAG API v1')

class Query(BaseModel):
    question: str
    session_id: str = 'default'

@app.post('/ask')
async def ask(query: Query) -> dict:
    try:
        answer = await rag_chain.ainvoke(query.question)
        return {'answer': answer, 'session': query.session_id}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get('/stream')
async def stream_answer(question: str):
    async def generate():
        async for token in rag_chain.astream(question):
            yield f'data: {token} '
        yield 'data: [DONE] '
    return StreamingResponse(generate(), media_type='text/event-stream')

@app.post('/index')
async def index_document(url: str) -> dict:
    from langchain_community.document_loaders import WebBaseLoader
    docs = WebBaseLoader(url).load()
    chunks = splitter.split_documents(docs)
    vectorstore.add_documents(chunks)
    return {'indexed': len(chunks)}

# uvicorn main:app --reload --port 8000
```

Cost & Performance Optimization

Cost Reduction

- Use gpt-4o-mini first (10x cheaper)
- Semantic cache repeat queries
- Reduce k from 8 to 3-4 chunks
- Compress chunks before sending
- Batch with .batch() not loops
- Count tokens before calling API

Quality Improvement

- Add cross-encoder reranking
- Use hybrid BM25+vector retrieval
- Tune chunk size per doc type
- Filter low-score chunks (<0.75)
- Expand queries with MultiQuery
- Measure with RAGAs regularly